

A Metamodel for AI-Agent Telemetry: Assessing Diagnostic Performance

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, CA, United States
krishna.balusu@outlook.com

Abstract

Autonomous AI agents built on large language models are entering production, but the observability infrastructure that monitors them remains constrained by legacy metamodels. Generic OpenTelemetry was designed for request-response microservices; the AI-specific conventions OpenTelemetry GenAI and OpenInference introduce typed agent and tool entities but lack typed sub-operations for delegation topology, planning, reasoning, memory, and constrained guardrail outcomes. We present a Domain-Specific Metamodel (DSM) for AI-agent telemetry: an Ecore abstract syntax with nine semantic span kinds, six OCL well-formedness invariants, and a W3C-compatible context propagation profile. Aligning with the workshop’s special theme of assessing MDE performance, we conduct a constructive evaluation of observability *expressivity* on a small synthetic benchmark of fourteen agent-specific fault classes (thirteen mapped onto the MAST taxonomy of Cemri et al.) across six telemetry conditions, seven agent frameworks, and six mocked foundation models; the foundation-model dimension contributes no independent variance, so the effective evaluation is a deterministic coverage matrix over fault, framework, and telemetry schema. Vanilla OTel, OTel GenAI, and OpenInference all converge to FDR 0.429 *under typed-only comparison*; the DSM averages 0.612 across all seven adapters (FPR 0.071, due to the infinite_loop detector misfiring on legitimately repeated CrewAI tool calls) and reaches a constructive upper bound of 1.000 on the conformance-complete reference adapter. The result is typed-metamodel *expressivity*, not production diagnostic performance. The artifact (Ecore, OCL, fault→element table, coverage matrix, MAST mapping, pinned environment) is released with the paper.

1 Introduction

Autonomous agents built on large language models (LLMs) are no longer toy projects; they execute production workflows in customer support, code generation, and incident response [10, 13, 25]. Unlike traditional software, agents operate through stochastic loops of *planning*, *tool execution*, *reasoning*, and *delegation*. As they enter production, observability has become critical, and a growing body of evidence shows that multi-agent failures are dominated by *agent-specific* pathologies (system design, inter-agent misalignment, task verification) rather than the network errors that dominate microservice incidents [9].

Observability is fundamentally a metamodeling problem [2, 3, 6]. A telemetry stack is a metamodel that fixes *what* execution data is captured, *how* it is categorized, and *how* disparate traces are graphed. The de-facto standard, OpenTelemetry (OTel), defines a metamodel for distributed traces whose abstract syntax centers on Span entities classified by a small SpanKind enumeration

(CLIENT, SERVER, INTERNAL, PRODUCER, CONSUMER). It was designed for request-response microservices.

Two AI-specific conventions extend OTel: OTel GenAI [18] introduces nine operation names that include chat, create_agent, invoke_agent, invoke_workflow, execute_tool, and retrieval; OpenInference [1] introduces ten typed span kinds spanning LLM, embedding, chain, retriever, reranker, tool, agent, guardrail, evaluator, and prompt. Both narrow the gap relative to vanilla OTel. Neither, however, provides typed sub-operations for the cognitive and coordination phases that distinguish agent workflows: *planning*, *reasoning*, peer-agent *delegation* with typed source/target identifiers, *memory* operations, and constrained guardrail outcomes. In both, “invoke an agent” or “invoke a workflow” is an opaque LLM-framed wrapper.

We address this gap by applying Model-Driven Engineering principles to the observability schema for autonomous agents. Our contributions are:

- (C1) A **Domain-Specific Metamodel** (DSM) defined as an Ecore-style abstract syntax (Fig. 1) with nine semantic SpanKind entities, six OCL well-formedness invariants (Listing 1), and a W3C-compatible context propagation profile (§4).
- (C2) A **constructive performance assessment** of the DSM against generic OTel, OTel GenAI (with the current retrieval, create_agent, and invoke_agent operations), and OpenInference (six of ten typed kinds exercised) on fourteen agent-specific fault classes mapped onto MAST [9], over 3,528 fault-injection runs plus 252 fault-free controls (§5). All numbers are reproduced from the artifact.
- (C3) A **conformance analysis** (§6) that separates *metamodel expressivity* (decisive: only the DSM can express the eight coordination/cognitive/policy faults) from *adapter conformance* (with a 3/9-to-7/9 coverage spread across third-party frameworks), and motivates conformance-driven adapter generation as the next research direction.

2 Related Work

MDE for AI. The MDE Intelligence series has applied MDE to AI artifacts: instance model generation from LLMs [19], syntactic-validity guarantees via grammar masking [15], DSML construction with LLM assistance [11], conformance preservation under LLM-driven model edits [14], and modeling AI-driven workflows [20]. Our work continues this MDE-for-AI direction but targets the *run-time* artifact (telemetry traces) rather than design-time artifacts. Cámara et al. [8] evaluate generative AI on modeling tasks; we conversely apply modeling techniques to evaluate observability of AI [7].

AI agent observability. Three industry vocabularies compete: OpenTelemetry GenAI [18] (operation-name oriented), OpenInference [1] (kind-oriented; ten typed kinds including AGENT, GUARDRAIL, RETRIEVER, TOOL), and proprietary stacks (LangSmith, Phoenix). The closest to our DSM is OpenInference, which lifts AGENT and GUARDRAIL to typed spans; however, its AGENT is a coarse wrapper without typed sub-operations for planning, reasoning, delegation, or memory, and its GUARDRAIL carries free-form attributes rather than a constrained outcome. Our DSM splits AGENT into seven sub-kinds plus typed memory, gives Delegation explicit source/target identifiers, and binds the whole with OCL invariants. *We benchmark all three baselines empirically (§5).*

Telemetry metamodeling and runtime models. OpenTelemetry Schema URLs / Schema Files version a telemetry vocabulary as a first-class artifact; DMTF’s CIM [12] does the same for system monitoring. The MDE community has long studied *models@runtime* [4] and model-based monitoring; production agent observability stacks (LangSmith, Arize Phoenix, OpenInference) are essentially industry instances with implicit (untyped) trace metamodels. We make the trace metamodel explicit, typed, and OCL-constrained.

Metamodel evaluation. Bork et al. [5] survey modeling-language specification techniques and call for systematic evaluation. Whittle et al. [24] document the gap between MDE practice and MDE evaluation. Our use of FDR as a constructive expressivity probe is a step toward such evaluation for the observability sub-domain.

3 The Semantic Gap

Consider the lifecycle of a single agent task: receive a user prompt, decompose it into a plan, query an LLM to reason over the plan, invoke a code-interpreter tool, delegate a sub-task to a researcher agent, retrieve grounded context, and pass the final output through a safety guardrail. Table 1 contrasts how four observability meta-models — generic OTel, OTel GenAI, OpenInference, and our DSM — categorize each step.

Table 1: Semantic information loss across observability meta-models.

Operation	V-OTel	OTel GenAI	Open-Inference	DSM (ours)
LLM call	CLIENT	chat	LLM	LLM_CALL
Tool exec.	CLIENT	execute_tool	TOOL	TOOL_CALL
Plan gen.	INTERNAL	chat	CHAIN	PLANNING
Reasoning step	INTERNAL	chat	CHAIN	REASONING
Delegation	CLIENT	invoke_agent [†]	AGENT [†]	DELEGATION
RAG retrieval	CLIENT	retrieval	RETRIEVER	RETRIEVAL
Guardrail	INTERNAL	—	GUARDRAIL [‡]	GUARD_RAIL
Memory r/w	INTERNAL	—	—	MEMORY
Agent lifecycle	—	create_agent	AGENT	AGENT

[†] Opaque wrapper without typed parent/target identity. [‡] Free-form attributes; no constrained pass/fail/warn outcome.

4 A Domain-Specific Metamodel for AI Agent Observability

We define the DSM as a metamodel conforming to MOF [17], expressed in Ecore [21]; the machine-readable metamodel and

OCL invariants are released with the artifact (agenttelemetry.ecore, agenttelemetry.ocl). Figure 1 gives the abstract syntax. The root container is Trace, which owns AgentSpan instances. AgentSpan is abstract; nine concrete subclasses cover the agent operations (one per SpanKind). Each AgentSpan carries Attributes, BaggageEntry, a parentId, and an agentId. The Delegation subclass exposes sourceAgent and targetAgent as Ecore-derived fields (derived="true") that project the underlying generic Attribute entries with keys delegation.source_agent and delegation.target_agent; the OCL invariants read those same generic keys, so the structural fields and OCL operate on a single source of truth. The kind discriminator on AgentSpan is itself derived: a GenModel EAnnotation encodes the rule that kind equals the SpanKind literal of the concrete subclass.

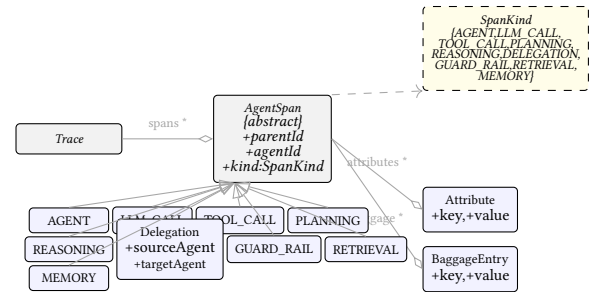


Figure 1: Ecore-style abstract syntax of the DSM.

4.1 Well-formedness constraints

Six OCL invariants [16] (Listing 1) accompany the metamodel. Five cover one distinct fault category each (two for coordination, one for topology, one for execution, one for policy); the sixth, KindMatchesSubtype, ties the redundant kind discriminator to the concrete subclass, closing the discriminator-vs-subclass loop-hole.

Listing 1: OCL well-formedness constraints on the DSM (released as agenttelemetry.ocl).

```

context Trace inv RootIsAgent:
self.spans
->select(s | s.parentId.oclIsUndefined())
->forAll(s | s.kind = SpanKind::AGENT)

context AgentSpan inv DelegationCarriesIdentity:
self.kind = SpanKind::DELEGATION implies (
  self.attributes->exists(a |
    a.key = 'delegation.source_agent') and
  self.attributes->exists(a |
    a.key = 'delegation.target_agent'))

context Trace inv NoCircularDelegation:
-- pairwise (2-cycle) check; longer-cycle SCC
-- handled by the runtime conformance checker.
let edges =
  self.spans->select(s | s.kind = SpanKind::DELEGATION)
->collect(s | Tuple {
  src = s.attributes->select(a |
    a.key='delegation.source_agent')
->first().value,
  tgt = s.attributes->select(a |
    a.key='delegation.target_agent')
->first().value }) in
edges->forAll(e1 | edges->forAll(e2 |

```

```

    not (e1.src = e2.tgt and e1.tgt = e2.src)))

context AgentSpan inv CostMetricsRequired:
  self.kind = SpanKind::LLM_CALL implies (
    self.attributes->exists(a | a.key='llm.input_tokens') and
    self.attributes->exists(a | a.key='llm.output_tokens') and
    self.attributes->exists(a | a.key='llm.cost'))

context AgentSpan inv GuardRailResultIsExplicit:
  self.kind = SpanKind::GUARD_RAIL implies
  self.attributes->exists(a |
    a.key = 'guardrail.result' and
    Set{'pass', 'fail', 'warn', 'bypass'}->includes(
      a.value.oclAsType(String)))

context AgentSpan inv KindMatchesSubtype:
  -- closes the discriminator-vs-subclass loophole.
  Sequence{Agent, LlmCall, ToolCall, Planning, Reasoning,
    Delegation, GuardRail, Retrieval, Memory}
  ->forAll(c | self.oclIsKindOf(c) implies
    self.kind = SpanKind::valueOf(c.name))

```

4.2 Inter-agent context propagation

The DSM uses two complementary identity mechanisms. Within a trace, topology is captured by typed source/target attributes on Delegation spans (delegation.source_agent / delegation.target_agent; see Listing 1). Across processes, the DSM extends W3C Trace Context [22] and Baggage [23] with two semantic baggage keys that propagate agent identity across HTTP and message-queue boundaries. The NoCircularDelegation invariant and the circular_delegation detector both read the in-trace pair; the run-time conformance checker validates that the baggage keys appear on every cross-process span downstream of a Delegation.

5 Assessing Metamodel Performance

In MDE, the performance of an observability metamodel is its *diagnostic utility*: how effectively does the metamodel let an automated rule decide whether a given trace contains a fault? We assess the DSM using Fault Detection Rate (FDR) paired with False-Positive Rate (FPR). We frame this as a *constructive evaluation of observability expressivity*: we measure whether each metamodel can in principle support detection of each fault class on the seven adapter+app pairs we built; we do not claim production diagnostic validation on independent workloads.

5.1 Methodology

Fault taxonomy. We constructed a taxonomy of fourteen agent-specific fault types and mapped each onto the multi-agent failure taxonomy (MAST) of Cemri et al. [9], whose three categories are *system design issues*, *inter-agent misalignment*, and *task verification*. Thirteen of fourteen faults map to MAST modes (artifact: mast_mapping.md); cost_explosion is our addition. Of MAST’s fourteen modes (FM-1.1 through FM-3.3), our taxonomy covers twelve distinct modes; the two unmapped modes — FM-1.2 (Disobey Role Specification) and FM-2.2 (Fail to Ask for Clarification) — are not observable from runtime telemetry alone. Six of the fourteen faults surface from generic OTel attributes alone (the “easy six”); the remaining eight require typed agent-specific spans (the “hard eight”).

Conditions. We test six telemetry conditions: (a) **No Telemetry** (control); (b) **Vanilla OTel**; (c) **OTel GenAI** (current spec: chat, create_agent, invoke_agent, execute_tool, retrieval, embeddings); (d) **OpenInference** (typed kinds AGENT, LLM, TOOL, CHAIN, RETRIEVER, GUARDRAIL); (e) **DSM (Metadata)** (nine kinds, prompts/completions redacted); (f) **DSM (Full)** (nine kinds, full payload).

Experimental matrix. We run all 14 faults $\times$ 6 conditions $\times$ 7 frameworks $\times$ 6 mocked foundation models = 3,528 fault-injection runs, plus $6 \times 7 \times 6 = 252$ fault-free control runs (for FPR), all on a deterministic seed. The model dimension does not contribute additional fault-detection variance under deterministic mocks; we retain it because it varies the cost-per-token magnitudes that drive the cost_explosion detector.

Detector. For each (condition, fault) pair we hand-coded a deterministic Python predicate over exported spans (one branch per fault, four banks for vanilla / GenAI / OpenInference / DSM). Predicates use only what the metamodel exposes. For instance, the DSM circular_delegation detector is one line — find a pair of DELEGATION spans whose (src,tgt) reverses — because the metamodel makes the topology explicit. The same predicate is impossible to write under the three baselines because none exposes typed (agent_id, target_id) on a typed delegation span. A trace is positive iff the predicate fires.

Fault $\rightarrow$ metamodel-element mapping. Table 2 gives the exact DSM element, OCL invariant (where applicable), and detector attribute that each fault relies on. Faults whose required element is absent in a baseline metamodel are unobservable in that baseline by construction.

Table 2: Each fault $\rightarrow$ required DSM element, OCL invariant (if any), and detector attribute. *Easy* = surfaces from generic LLM/tool attributes; *Hard* = requires agent-specific typed spans.

Fault	DSM element	OCL inv.	Detector attribute
<i>Easy six (detectable by all telemetry conditions):</i>			
wrong_tool	TOOL_CALL	–	tool.name
infinite_loop	TOOL_CALL	–	repeated tool.name
tool_failure	TOOL_CALL	–	OTel ERROR
timeout	any	–	ERROR+“timeout”
context_overflow	LLM_CALL	CostMetrics	llm.input_tokens
cost_explosion	LLM_CALL	CostMetrics	sum(llm.cost)
<i>Hard eight (detectable only with DSM-conformant traces):</i>			
circular_deleg.	DELEGATION	NoCircular	delegation.{src,tgt}_agent
hallucination	LLM_CALL, RETRIEVAL	–	no RETRIEVAL sibling
stale_retrieval	RETRIEVAL	–	retrieval.staleness_seconds
guardrail_byp.	GUARD_RAIL	GuardRail	guardrail.result
planning_failure	PLANNING	–	planning.step_count
reasoning_loop	REASONING	–	reasoning.chain
agent_misroute	AGENT, DELEG.	Deleg.Id.	agent.misrouted
memory_corrupt.	MEMORY	–	memory.corrupted

5.2 Results

DSM (Metadata) and DSM (Full) are identical because every detector operates on typed structure and metadata attributes, never on payloads — payload redaction is FDR-neutral, an explicit privacy benefit of the design. The DSM FPR of 0.071 originates from

Table 3: Aggregate FDR (over 3,528 fault-injection runs) and FPR (over 252 fault-free runs).

Condition	FDR	FPR	@ FDR=1
No Telemetry	0.000	0.000	0/14
Vanilla OTel	0.429	0.000	6/14
OTel GenAI (current spec)	0.429	0.000	6/14
OpenInference	0.429	0.000	6/14
DSM (Metadata)	0.612	0.071	7/14
DSM (Full)	0.612	0.071	7/14
<i>DSM, ref. adapter (upper bound)</i>	1.000	0.000	14/14

three CrewAI fault-free runs (out of 42 per condition; reproducible with the artifact’s `compute_fpr.py`) on which the `infinite_loop` detector misfires: the CrewAI multi-agent workload legitimately invokes the same tool-calculator three times across writer turns, exceeding the detector’s `count $\geq$ 3` threshold. We treat this as a real false-positive of the threshold-based detector against a non-fault repeated-tool pattern, not a metamodel defect; tightening the detector to require both repetition *and* no monotonic state progression is straightforward future work.

Scope: standardized typed semantics. The `otel_genai` and `openinference` detectors exploit only the typed kinds and standardized attributes each spec defines. The OpenInference benchmark app does emit free-form delegation attributes (`delegation.source_agent`, `delegation.target_agent`) on its AGENT child span; we exclude these from the OpenInference detector because they are not part of the OpenInference typed model. The comparison is between standardized typed semantics; a custom-attribute baseline would be isomorphic to a partial DSM and is left to future work.

Finding 1: All three baselines converge. Vanilla OTel, the corrected OTel GenAI, and OpenInference all reach 0.429. Each detects the same six faults — those surfacing as LLM-call or tool-call attributes. Despite GenAI providing `invoke_agent` and OpenInference providing AGENT/GUARDRAIL, none can detect any of the eight coordination/cognitive/policy faults: `invoke_agent` carries no source/target identifier, OpenInference AGENT is a coarse wrapper, and GUARDRAIL carries no constrained outcome. In this benchmark, under typed-only comparison, typed agent kinds without typed sub-operations close none of the diagnostic gap.

Finding 2: Metamodel expressivity is necessary but not sufficient. The DSM is the only metamodel that *can* express the failing eight fault classes. On the reference `custom_agent` (Listing 1-conformant), the DSM reaches FDR = 1.000 across all 14 faults. Aggregated across all seven adapter+app pairs, the DSM averages 0.612: the six easy faults at 1.000 plus heterogeneous partial credit on the eight hard faults (Table 4). The 1.000 result is a constructive upper bound (the reference adapter and the detectors share authorship), demonstrating expressivity rather than independent empirical validation.

Finding 3: The remaining gap is a per-app conformance problem. Per-app DSM coverage in the seven benchmark apps spans 3/9 to 9/9 (artifact: `coverage_matrix.md`): `custom_agent` 9/9, `llamaindex_agent` 7/9, `langchain_rag` 6/9, `autogen_group`

5/9, `crewai_team` 4/9, `anthropic_sdk` and `openai_sdk` 3/9. All seven emit typed DELEGATION when the workload requires it, so `circular_delegation` reaches FDR = 1.000 across all apps. Only `custom_agent` emits GUARDRAIL and MEMORY with conformant attributes (and `retrieval.staleness_seconds`) — the six remaining hard faults collapse to FDR = 0.143 (1/7). The exception is hallucination: five of seven apps detect it (FDR = 0.71), because the no-grounding heuristic fires on a long LLM_CALL output without a sibling RETRIEVAL; LangChain and LlamaIndex emit non-conformant retrieval annotations under our injection that defeat the heuristic.

Table 4: Per-fault FDR. DSM is the all-frameworks aggregate (0.14 = 1/7 framework conformance); DSM* restricts to the conformance-complete custom_agent reference adapter and is reported as a constructive upper bound.

Fault	None	V-OT	GenAI	OI	DSM	DSM*
wrong_tool	0.00	1.00	1.00	1.00	1.00	1.00
infinite_loop	0.00	1.00	1.00	1.00	1.00	1.00
tool_failure	0.00	1.00	1.00	1.00	1.00	1.00
timeout	0.00	1.00	1.00	1.00	1.00	1.00
context_overflow	0.00	1.00	1.00	1.00	1.00	1.00
cost_explosion	0.00	1.00	1.00	1.00	1.00	1.00
circular_deleg.	0.00	0.00	0.00	0.00	1.00	1.00
hallucination	0.00	0.00	0.00	0.00	0.71	1.00
stale_retrieval	0.00	0.00	0.00	0.00	0.14	1.00
guardrail_bypass	0.00	0.00	0.00	0.00	0.14	1.00
planning_failure	0.00	0.00	0.00	0.00	0.14	1.00
reasoning_loop	0.00	0.00	0.00	0.00	0.14	1.00
agent_misroute	0.00	0.00	0.00	0.00	0.14	1.00
memory_corruption	0.00	0.00	0.00	0.00	0.14	1.00

5.3 Threats to validity

Construct. FDR is a binary detection metric; it does not capture diagnostic *quality*. We restrict positive signal to deterministic predicates that name the exact fault, partially mitigating this. We additionally report FPR.

Internal (constructive bias). The fourteen-fault taxonomy, the nine DSM kinds, the OCL invariants, the detectors, and the reference `custom_agent` are all authored by us, so the 1.000 result is a *constructive upper bound* demonstrating expressivity rather than independent empirical validation. We mitigate by mapping the fault taxonomy onto MAST and benchmarking three independent industry baselines (Vanilla OTel, OTel GenAI, OpenInference). The honest empirical claim is twofold: (a) the DSM is necessary — no metamodel without typed coordination/cognitive/policy spans detected any of the eight hard faults — and (b) at least one prototype adapter realizes the metamodel. We do not claim external validity beyond these synthetic benchmark scenarios; production diagnostic validation on independent third-party traces is future work.

Statistical reporting. All numbers are exact under deterministic mocks (single seed, fixed RNG); they are not point estimates with sampling variance. A stochastic re-evaluation under multiple seeds with 95% CIs and real LLM endpoints is the next milestone.

Aggregation. Reporting an arithmetic mean over seven frameworks, several at 0.000 on the eight hard faults, can inflate the

apparent score. We accordingly report both the aggregate (Table 3) and the per-fault breakdown (Table 4); the per-framework breakdown is in the artifact.

6 Conformance and the Adapter-Generation Direction

The split between metamodel expressivity (1.000 on the conforming app) and aggregated FDR (0.612) is the actionable result. It localizes the bottleneck to a well-known MDE problem: instance conformance to a metamodel [2]. Each adapter is, in MDE terms, a model-to-model transformation from framework events to DSM-conformant spans. Three follow-up directions: (i) LLM-assisted adapter synthesis seeded by the metamodel as a target schema, e.g. a grammar-masked prompt [15] constraining the model to emit only constructors that satisfy Listing 1; (ii) runtime conformance checking that loads `agenttelemetry.ocl` into Eclipse OCL or USE and validates exported traces against all six invariants; and (iii) a state-aware OCL constraint that distinguishes legitimate retries from pathological loops, motivated by the present CrewAI false positive.

7 Reproducibility

Released as a replication artifact: the machine-readable Ecore metamodel and OCL invariants (`agenttelemetry.ecore / .ocl`), benchmark code, fault injectors, per-app coverage matrix, MAST mapping, an FPR script, per-run results (3,780 rows), and a pinned environment (`requirements.lock`, Python 3.11+; full matrix runs in ≈ 3 min on one CPU). Tables 3 and 4 reproduce via:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.lock && pip install -e .
PYTHONPATH=src:. python benchmarks/run_benchmarks.py \
--output benchmarks/results_full.tsv
PYTHONPATH=src:. python benchmarks/compute_fpr.py \
benchmarks/results_full.tsv
```

Artifact: <https://github.com/Krishnachaitanyakc/AgentTelemetry>.

8 Conclusion

Generic microservice observability metamodels lose the semantic information that AI agent diagnosis requires; the emerging AI-specific conventions (OTel GenAI with current agent operations, OpenInference with typed kinds) do not close the diagnostic gap on the eight coordination/cognitive/policy faults that dominate the multi-agent failure literature: in our constructive evaluation, all three baselines converge at FDR 0.429. The DSM raises FDR to 0.612 across third-party apps and to a constructive upper bound of 1.000 on a conformance-complete reference adapter. We frame this as observability *expressivity*, not production diagnosis: the empirical claim is that typed coordination, cognitive, and policy spans are *necessary* for these faults, and the open MDE problem is automated, conformance-driven adapter generation.

References

- [1] Arize AI. 2024. OpenInference: Semantic Conventions for AI System Observability. <https://github.com/Arize-ai/openinference>.
- [2] Colin Atkinson and Thomas Kühne. 2003. Model-Driven Development: A Meta-modeling Foundation. *IEEE Software* 20, 5 (2003), 36–41. doi:10.1109/MS.2003.1231149
- [3] Jean Bézivin. 2005. On the unification power of models. *Software and Systems Modeling* 4, 2 (2005), 171–188. doi:10.1007/s10270-005-0079-0
- [4] Gordon Blair, Nelly Bencomo, and Robert B. France. 2009. Models@run.time. *Computer* 42, 10 (2009), 22–27. doi:10.1109/MC.2009.326
- [5] Dominik Bork, Dimitris Karagiannis, and Benedikt Pittl. 2020. A survey of modeling language specification techniques. *Information Systems* 87 (2020), 101425. doi:10.1016/j.is.2019.101425
- [6] Marco Brambilla, Jordi Cabot, and Manuel Wimmer. 2017. *Model-Driven Software Engineering in Practice* (2 ed.). Morgan & Claypool. doi:10.2200/S00751ED2V01Y201701SWE004
- [7] Lola Burgueño, Alexandru Burdusel, Sébastien Gérard, and Manuel Wimmer. 2022. Guest editorial to the theme section on AI-enhanced model-driven engineering. *Software and Systems Modeling* 21 (2022), 963–965. doi:10.1007/s10270-022-00988-0
- [8] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. 2023. On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling* 22 (2023), 781–793. doi:10.1007/s10270-023-01105-5
- [9] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657* (2025).
- [10] Harrison Chase. 2023. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>.
- [11] Claudio Di Sipio, Riccardo Rubel, Juri Di Rocco, Davide Di Ruscio, and Ludovico Iovino. 2024. On the use of LLMs to support the development of domain-specific modeling languages. In *Proc. MDE Intelligence 2024 (MODELS Companion)*.
- [12] Distributed Management Task Force. 2024. Common Information Model (CIM) Specification. <https://www.dmtf.org/standards/cim>.
- [13] João Moura. 2023. CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://github.com/crewAIInc/crewAI>.
- [14] Lukas Netz, Finn Kampe, Jan Reimer, and Bernhard Rumpe. 2025. Unintended Changes: How LLMs Corrupt and Correct Textual Models. In *Proc. MDE Intelligence 2025 (MODELS Companion)*.
- [15] Lukas Netz, Jan Reimer, and Bernhard Rumpe. 2024. Using Grammar Masking to Ensure Syntactic Validity in LLM-based Modeling Tasks. In *Proc. MDE Intelligence 2024 (MODELS Companion)*.
- [16] Object Management Group. 2014. Object Constraint Language (OCL), v2.4. <https://www.omg.org/spec/OCL/2.4/>.
- [17] Object Management Group. 2019. Meta Object Facility (MOF) Core Specification, v2.5.1. <https://www.omg.org/spec/MOF/2.5.1/>.
- [18] OpenTelemetry Contributors. 2024. OpenTelemetry GenAI Semantic Conventions. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>.
- [19] Fengjunjie Pan, Nenad Petrovic, Vahid Zolfaghari, Long Wen, and Alois Knoll. 2025. LLM-enabled Instance Model Generation. In *Proc. MDE Intelligence 2025 (MODELS Companion)*.
- [20] Tiago Sousa, Nicolas Guelfi, and Benoît Ries. 2025. Modeling AI-Driven Workflows for Ecosystem Resilience Prediction. In *Proc. MDE Intelligence 2025 (MODELS Companion)*.
- [21] Dave Steinberg, Frank Budinsky, Marcelo Paternostro, and Ed Merks. 2008. *EMF: Eclipse Modeling Framework* (2 ed.). Addison-Wesley.
- [22] W3C Distributed Tracing Working Group. 2021. Trace Context. <https://www.w3.org/TR/trace-context/>. W3C Recommendation, 23 November 2021.
- [23] W3C Distributed Tracing Working Group. 2024. Baggage. <https://www.w3.org/TR/baggage/>. W3C Candidate Recommendation Snapshot, 30 May 2024.
- [24] Jon Whittle, John Hutchinson, and Mark Rouncefield. 2014. The State of Practice in Model-Driven Engineering. *IEEE Software* 31, 3 (2014), 79–85. doi:10.1109/MS.2013.65
- [25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155* (2023).